



BriteTest

Runner Internal Reference

Document Version: 1.0

Runner Version: 1.0.0

Test Version: 1.0.0

This document provides a reference to the BriteTestRunner API internals. It includes types, structs, unions, enums, macros, and functions.

Copyright (c) 2026 Paul Sinclair

License SPDX-License-Identifier: MIT.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Preface

For a list of other BriteTest documents and the BriteTest repository layout, see the Documentation Guide.

For a glossary of terms, see the Glossary Reference.

Document Version History

Document	Runner	Test	Date	Comment	Author/Editor
1.0	1.0.0	1.0.0	2026-06-11	Initial version.	Paul Sinclair

- The **Document** column records the document's version with the format **M.u** (Major, update).
- The **Runner** column records the Runner API version current at the time this document version was published and is defined by its `RA_RUNNER_VERSION` macro.
- The **Test** column records the Test API version current at the time this document version was published and is defined by its `RA_TEST_VERSION` macro.
- Both Runner and Test use the version format "**M.m.p**" (Major, minor, patch).
- **M** is the same for the Document, Runner, and Test versions.

The document's update version tracks released updates to this document and does not correspond to a minor or patch version. **u** increments when document changes are published in a release without a change to **M**, and it resets to **0** when **M** is incremented.

Table of Contents

Introduction	1
Public and Internal Name Conventions	??
Symbols Defined in ‘runnerapi.h’	2
Types	2
Structs	2
Enums and Enum Values	3
Global Variables	3
Macros	3
Functions	4
Symbols Defined in ‘runnerapi.c’	5
Types	2
Structs	2
Enums and Enum Values	3
Global Variables	3
Macros	3
Functions (declared as a forward reference) in ‘runnerapi.h’	??
Functions (not declared in ‘runnerapi.h’)	??
Execution Engine	6
Signal Handling	7
Process Management	8
Thread Management	9
File and Path Support	10
Matching and Comparison Support	11
Environment Support	12
Glossary	14

1. Introduction

BriteTest's internal architecture consists of several cooperating subsystems, including the implementation of API macros and functions, the execution engine, guard/fault handling, isolation support, file/path utilities, matching/comparison helpers, and environment support. These components work together to run test groups and test expressions reliably across threads and processes while capturing faults and producing structured reports.

The remainder of this document describes each internal symbol used to implement these subsystems, organized from high-level behavior down to low-level details.

In this document, a *symbol* refers to any named entity in the BriteTest framework, including:

- typedefs
- structs
- enums and enum values
- macros
- variables
- functions

This document is organized by the named entities in the BriteTest Runner implementation. Each symbol (type, macro, function, etc.) is described individually, including its purpose, behavior, and usage. It serves as the reference companion to the BriteTest Framework Guide, defining the framework's components precisely while the Guide explains their design and interaction.

###1.2. Public and Internal Naming Conventions

Any framework names that are public and visible to BriteTest API users are prefixed with `ra_...` (typically lowercase) or `RA_...` (typically uppercase).

Any framework names that are internal (but technically visible to BriteTest API users) follow the pattern `britetest_..._internal_t`, `britetest_..._internal`, or `BRITETEST_..._INTERNAL`. These names should not be referenced by API users.

In general, users of the API should not define names prefixed with `ra_`, `BT`, `britetest_`, or `BRITETEST`, or reference names prefixed with `britetest_` or `BRITETEST_`.

2. Symbols Defined in runnerapi.h

These are used by `runnerapi.h` and `runnerapi.c`, and are public or internal based on their name per the naming conventions.

2.1. Types

These typedefs declare fundamental internal types used throughout the BriteTest framework.

ra_resubt_t Declaration

```
typedef struct
{
    size_t pass;
    size_t fail;
    size_t fault;
    size_t injected_fail;
    size_t injected_fault;
} ra_resubt_t;
```

Description

Result counters produced by test execution and propagated through group and orchestrator aggregation logic.

Usage Notes

- `fault == SIZE_MAX` is a sentinel meaning a function-level fault was captured.
- Function-level fault sentinel values are introduced by the signal-guard path.
- This sentinel is distinct from ordinary fault counts returned by test logic.
- Related helpers include `ra_currentresult(void)` and result-merging macros.

Example

```
ra_resubt_t result = {0, 0, 0, 0, 0};
```

2.2. Structs

struct Declaration

```
typedef struct <StructName> {
    <field-type> <field-name>;
    ...
} <StructName>;
```

Description

Explain the purpose of this struct, what data it aggregates, and how it participates in the BriteTest execution model.

Fields - <field-name> — description

- <field-name> — description

Usage Notes

- Ownership or lifetime rules
- Whether fields must be initialized by the user or framework

Example

```
<StructName> s = { ... };
```

2.3. Enums and Enum Values

enum Declaration

```
typedef enum <EnumName> {  
    <ENUM_VALUE_1>,  
    <ENUM_VALUE_2>,  
    ...  
} <EnumName>;
```

Description

Explain what conceptual category this enum models and how the values are used by the framework.

Values - <ENUM_VALUE_1> — meaning

- <ENUM_VALUE_2> — meaning

Usage Notes

- Any ordering assumptions
- Whether values map to external formats (strings, logs, etc.)

Example

```
<EnumName> mode = <ENUM_VALUE_1>;
```

2.4. Global Variables

Declaration

```
extern <type> <VariableName>;
```

Description

Explain the purpose of this global variable, what state it represents, and how it is used by the BriteTest framework.

Usage Notes

- Whether the variable is read-only or writable
- Whether users are expected to modify it
- Thread-safety considerations
- Lifetime and initialization rules

Example

```
if (<VariableName> == ...) {  
    ...  
}
```

2.5. Macros

Definition

```
#define <MACRO_NAME>(...) <expansion>
```

Description

Describe the purpose of this macro, what it expands to conceptually, and how it fits into the BriteTest orchestration or test definition model.

Parameters - <param> — meaning and constraints

- <param> — meaning

Usage Notes

- Side effects
- Evaluation rules (e.g., multiple evaluation hazards)
- Whether arguments must be constant expressions

Example

```
<MACRO_NAME>(arg1, arg2);
```

2.6. Functions

The functions in this section may be **static**, **static inline**, or (by default) **extern**, depending on how they are used within the framework.

() Signature

```
<return-type> <FunctionName>(<parameters>);
```

Description

Explain what this function does, when it is called, and how it interacts with the BriteTest runtime.

Parameters - <param-name> — meaning, constraints, ownership

- <param-name> — meaning

Return Value

Describe what is returned and under what conditions.

Errors / Preconditions

- Preconditions the caller must satisfy
- Error conditions or undefined behavior cases

Usage Notes

- Thread safety
- Lifetime rules
- Interaction with other BriteTest components

Example

```
<return-type> result = <FunctionName>(...);
```

3. Symbols Defined in `runnerapi.c`

These symbols are local to `runnerapi.c` unless specified or defaulting to extern. Symbols that are local do not have to conform to the internal naming conventions and more natural names may be used.

3.1. Types

(Placeholder)

3.2. Structs

(Placeholder)

3.3. Enums and Enum Values

(Placeholder)

3.4. Global Variables

(Placeholder)

3.5. Macros

(Placeholder)

3.6. Functions (declared as a forward reference) in `runnerapi.h`

These functions are referenced internally by `runnerapi.h` but defined in `runnerapi.c`. These functions are (by default) extern and must conform to the internal name conventions.

(Placeholder)

3.7. Functions (not declared in `runnerapi.h`)

These functions are local to `runnerapi.c` and defined as static. Since these are local, these names do not have to conform to the internal name conventions and more natural names may be used.

(Placeholder)

4. Execution Engine

(Placeholder for execution helpers.)

5. Signal Handling

(Placeholder for signal guard helpers.)

6. Process Management

(Placeholder for fork/exec/wait logic.)

7. Thread Management

(Placeholder for pthread logic.)

8. File and Path Support

(Placeholder for filesystem support helpers.)

9. Matching and Comparison Support

(Placeholder for comparison support helpers.)

10. Environment Support

(Placeholder for environment support helpers.)

Repository Layout

GitHub repository: paulsinclair51/BriteTest

Repository layout (listing core files):

```
BriteTest/  
|- .github/  
|  \- workflows/  
|     \- ci.yml  
|- README.md  
|- LICENSE  
|- Makefile  
|- build_test_britetest.ps1  
|- build/  
|- docs/  
|  \- BriteTest_API_User_Guide.md  
|  \- BriteTest_API_Reference.md  
|  \- Contributor_Guide.md  
|  \- BriteTest_Framework_Guide.md  
|  \- BriteTest_Framework_Reference.md  
|- examples/  
|- include/  
|  \- runnerapi.h  
|- reports/  
|  |- test_report-I.txt  
|  \- test_report.txt  
|- scripts/  
|- src/  
|  \- runnerapi.c  
|- tests/  
|  |- test_runner.c  
|  |- orchestrator_tests.c  
|  |- guard1_tests.c  
|  \- guard2_tests.c
```

Use this as a reference when adapting BriteTest into your own project structure.

Glossary

Framework-Specific Terms

- **Orchestrator Lifecycle:** The sequence of initialization, group execution, test execution, and report finalization performed by the BriteTest framework.
- **Guard Behavior:** The mechanism BriteTest uses to catch runtime faults (e.g., segmentation faults) and continue executing remaining tests.
- **Isolation Semantics:** The rules governing how tests and groups run in threads or processes to prevent interference and ensure fault containment.
- **Concurrency Model:** The framework's rules for running tests concurrently within `RA_BEGIN_CONCURRENT` / `RA_END_CONCURRENT` blocks.
- **Execution Phases:** The internal stages of orchestrator operation, including initialization, argument parsing, group dispatch, test dispatch, and report writing.
- **Fault Handling Model:** The framework's strategy for capturing and reporting faults without terminating the entire test run.
- **Control File Promotion:** The process of replacing an outdated control file with a newly generated file when differences are expected or intentional.
- **Nested Group Behavior:** The rules governing how groups may contain other groups and how isolation and concurrency propagate through nested structures.
- **Concurrent Block Behavior:** The semantics of executing multiple tests in parallel within a concurrent block, including ordering and isolation rules.